Interactive data visualizations for US communities crime exploration

Fabio Dollaku, HPDAV Individual Assignment, March 2026

1. Problem, data, and user tasks

I built an interactive visualization system for the US Communities and Crime dataset from the UCI Machine Learning Repository. The dataset has 1,355 communities across 46 US states, with over 100 normalized socio-economic and crime attributes. Some values are missing (marked as "?"), so I filter those out during rendering.

There are four things I wanted users to be able to do: compare any two continuous attributes across communities, see geographic patterns by grouping communities under states, select subsets of interest with a brush, and drill into specific states or communities through a hierarchy. The app runs on React 18, D3.js 7, and Redux Toolkit for keeping the two views in sync.

2. Visual design and justification

The layout is split into two linked panels: a scatterplot on the left and a hierarchical view on the right.

The scatterplot maps two user-selected attributes to x and y position, which is a straightforward way to compare continuous values. Users can switch axes via dropdowns. I added a D3 brush rectangle for multi-select: dragging over an area selects all the points inside it. Selected points go to full opacity with a red stroke, while everything else fades to 0.3 opacity. This makes it easy to pick out a subset and immediately see where those communities fall in the hierarchy.

For the hierarchical view, communities are grouped by state (two levels: State then Community). I tried three different layouts to see which one works best for this data:

Layout	What works	What doesn't
Treemap	Uses space well; area and color both encode the value, so you get two channels at once	The hierarchy itself is less obvious
Circle packing	Clear grouping; looks nice; containment is intuitive	Wastes a lot of space between circles
Tree (node-link)	You can actually see the parent-child structure clearly	Bad for comparing values; spreads out too wide

I went with the treemap as the default because it gives you two encodings at once: cell area shows relative size and a yellow-to-red color scale (d3.interpolateYlOrRd) shows the attribute value. Large dark-red cells jump out immediately, so you can spot high-crime communities within each state at a glance. States have gray borders with labels at the top of each region. The other two layouts are still available via dropdown for when you want a different perspective.

Figure 1: Treemap layout overview

Figure 2: Brush selection with synchronized highlighting

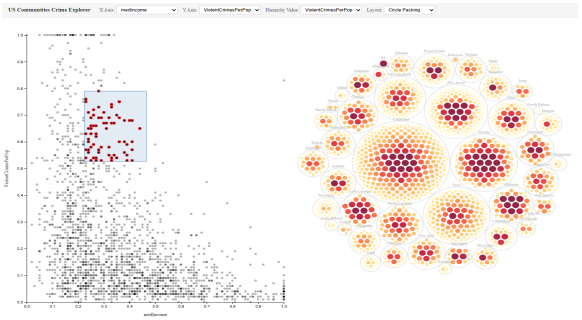

Figure 3: Circle packing layout

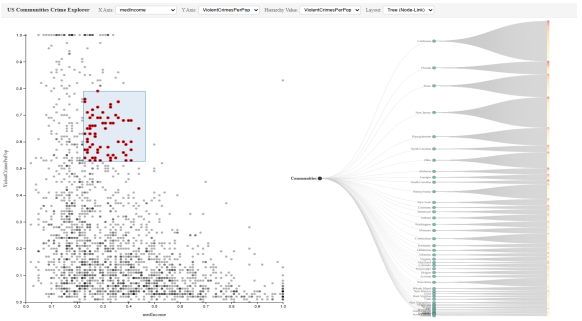

Figure 4: Tree (node-link) layout

3. Synchronized interactions

The two views talk to each other through a shared Redux store. When you brush a region on the scatterplot, the indices of the selected communities get dispatched to the store. The hierarchy view picks that up and highlights the matching nodes (full opacity) while fading everything else. It works the other way too: clicking a node in the hierarchy sends that selection back to the store, and the scatterplot highlights the corresponding points with a red stroke.

In practice, this means you can brush a cluster of outliers in the scatterplot and instantly see which states they belong to in the treemap, or click on California in the hierarchy and see where its communities land in the attribute space. Dropdowns let you switch X/Y axes and the hierarchy value attribute independently, so you can explore different combinations without losing your selection. I animated the transitions (300 to 1000 ms depending on the action) to keep things from jumping around too abruptly.

Declaration of generative AI use

The work I am submitting is my own. I used Claude (by Anthropic) at various points during development, mostly as a reference tool. When I ran into issues with D3.js rendering or Redux state not syncing the way I expected, I used it to help me debug. I also looked up D3 API details through it, things like brush interactions, hierarchy layouts, and scale configurations. All the design decisions, from choosing coordinated views to picking the treemap as the default, came from what I learned in the HPDAV course. The AI was not used to write this report or generate whole chunks of code.